ENTERPRISE ARCHITECTURE & LOGIC SPECIFICATION

# YOLKFLOW™

## Complete Project Logic, Mathematical Accounting Engine & System Diagrams

### এম. এ. খালেক সরকার — ডিমের আড়ত ও পাইকারি হালখাতা স্বয়ংক্রিয় খতিয়ান ব্যবস্থা

**APPLICATION NAME**
YolkFlow (Next.js 16.3 + React 19)

**DOMAIN / INDUSTRY**
Wholesale Egg Commerce (Bangladesh)

**PRIMARY MERCHANT**
M.A. Khalek Sarker Wholesale Egg Depot

**DATABASE ARCHITECTURE**
Dual-Layer: Google Sheets API v4 + Mock JSON

**ARTIFICIAL INTELLIGENCE**
Gemini Vision OCR Multi-Model Cascade

**CORE ACCOUNTING**
Traditional Bengali Hal-Khata Equation

Document Version 1.0 • Comprehensive Technical Compendium

All 10 Architectural Vector Diagrams Included

# Table of Contents

This technical compendium provides an exhaustive analysis of the codebase, business rules, accounting formulas, data structures, and operational workflows of the YolkFlow application.

**Chapter 1:** Executive Summary & Domain Overview Page 3

**Chapter 2:** High-Level System Architecture & Technology Stack Page 4

• *Diagram 1: End-to-End System Architecture* Page 5

**Chapter 3:** Component Hierarchy & Reactive UI State Machine Page 6

• *Diagram 2: Component Tree & State Interactions* Page 7

**Chapter 4:** Dual-Layer Persistence & Cloud Synchronization Engine Page 8

• *Diagram 3: Persistence Read/Write Pipeline & Sheet Duplication* Page 9

**Chapter 5:** Entity-Relationship Modeling & Data Schema Page 10

• *Diagram 4: Entity-Relationship Diagram (ERD) & Schema* Page 11

**Chapter 6:** Hal-Khata Mathematical Accounting Engine & Ledger Equations Page 12

• *Diagram 5: Core Hal-Khata Balance Equation & Profit Engine* Page 13

**Chapter 7:** Daily Transaction Lifecycle & Hal-Khata Entry Workflow Page 14

• *Diagram 6: Daily Ledger Entry & Data Lifecycle Workflow* Page 15

**Chapter 8:** AI Vision OCR Ingestion Pipeline (Bengali Handwriting Extraction) Page 16

• *Diagram 7: Gemini Vision OCR Multi-Model Ingestion Pipeline* Page 17

**Chapter 9:** Overhead Expenses, Fund Settlements & Liquidity Reconciliation Page 18

• *Diagram 8: Overhead Expenses & Cash Reconciliation Flow* Page 19

**Chapter 10:** Role-Based Access Control (RBAC) & Security Architecture Page 20

• *Diagram 9: RBAC Architecture & Permission Matrix* Page 21

**Chapter 11:** Real-time Business Intelligence, Analytics & Charting Engine Page 22

• *Diagram 10: Real-time Analytics & Spline Charting Engine* Page 23

**Chapter 12:** Production Readiness, Fault Tolerance & Recommendations Page 24

**Appendix:** Mathematical Index, Google Sheets Cells & API Specifications Page 25

> **Architectural Scope:** This document maps directly to the implementation inside `d:\eggapp`, referencing `src/lib/db.ts`, `src/lib/googleSheets.ts`, `src/lib/ledger.ts`, `src/lib/auth.ts`, `src/app/actions.ts`, `src/app/YolkFlowClient.tsx`, `src/app/OverheadExpensesView.tsx`, `src/app/ImageToJsonModal.tsx`, and all API route handlers.

# Chapter 1: Executive Summary & Domain Overview

## 1.1 The Business Domain: Bengali Wholesale Egg Trading

Wholesale egg distribution in Bangladesh is a high-volume, rapid-turnover commodity business characterized by volatile daily procurement prices, perishable inventory, tight profit margins (often pennies per egg), and widespread credit sales ("বাকি") to retail grocery stores, bakeries, and local markets. Traditional merchants ("আড়তদার") maintain physical paper notebooks called **দৈনিক হালখাতা** (Daily Hal-Khata). Every day, a ledger page is written recording four key sections: closing inventory stock, customer collections, opening/supplier debt, and day-to-day operating expenses.

## 1.2 Core Challenges of the Traditional System

- **Calculation Complexity:** Balancing the Hal-Khata requires multi-step manual arithmetic across stock valuations, customer credit collections, supplier payables, and operating expenses. Errors compound across consecutive days.
- **Inventory Shrinkage & Breakage ("ভাঙ্গা ডিম"):** Eggs cracked in transport or storage represent direct inventory and monetary leakage. Physical ledgers frequently fail to track cumulative waste percentages.
- **Single Point of Failure:** Physical ledger books are vulnerable to loss, water damage, and lack remote visibility for absentee business owners.
- **Disjointed Fund Management:** In-shop daily savings ("সমিতি"), bank DPS, and owner personal withdrawals ("নিজ") often mix with shop working capital, obscuring true profitability.

## 1.3 The YolkFlow Solution

YolkFlow digitizes wholesale egg accounting without disrupting the time-tested mental model of the Bengali merchant. Rather than forcing a generic western double-entry accounting software onto the merchant, YolkFlow adopts the exact structure of the physical Hal-Khata notebook, augmenting it with modern cloud synchronization, AI handwriting OCR, and executive analytics.

| CORE COMMODITY TYPES | CLOUD SYNC LATENCY | OCR SPEED |
|---|---|---|
| **6 Varieties** | **< 1.2s** | **< 3.5s** |
| White, Red, Duck, Chicken, Quail, LM | Bi-directional Google Sheets API v4 | Gemini Vision Multi-Model Cascade |

# Chapter 2: High-Level System Architecture

## 2.1 Architectural Paradigm: Modern Next.js Fullstack Engine

YolkFlow is architected as an enterprise fullstack web application powered by **Next.js 16.3.1 (App Router)**, **React 19.2.8**, and **TypeScript 5**. The architecture employs a tiered separation of concerns:

- **Client Presentation Tier:** Powered by React Server Components (RSC) and dynamic Client Components. Employs Tailwind CSS v4 and Lucide React icons, rendering an adaptive glassmorphic UI optimized for mobile, tablet, and desktop viewports.
- **Server Action & API Tier:** Utilizes Next.js Server Actions ( `actions.ts` ) for type-safe mutation pipelines with automatic router cache revalidation ( `revalidatePath("/")` ), alongside RESTful Route Handlers ( `/api/data` , `/api/overhead` , `/api/savings` , `/api/ocr-to-json` ).
- **Business Domain Services ( `src/lib` ):** Modular TypeScript services encapsulating domain mathematical equations ( `ledger.ts` ), multi-tier authentication ( `auth.ts` ), hybrid database operations ( `db.ts` ), and Google Sheets OAuth2 service account integration ( `googleSheets.ts` ).
- **Hybrid Persistence Layer:** Combines Google Sheets as a human-auditable cloud master with an atomic local JSON cache ( `mock_db.json` ) that guarantees zero downtime when operating offline.

## 2.2 Architectural Flow Diagram

The following vector diagram details the four operational tiers and the secure communication protocols connecting them.

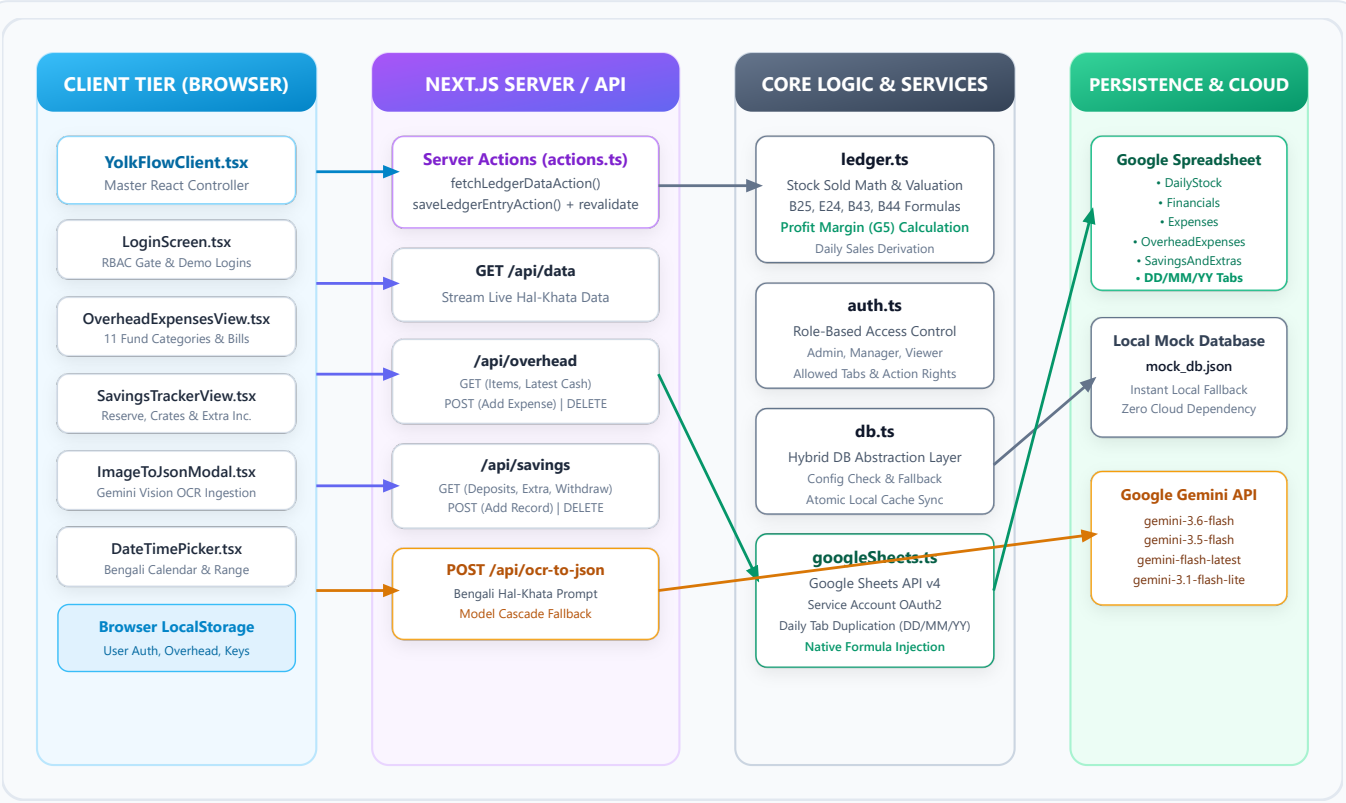



FIGURE 2.1: YOLKFLOW END-TO-END SYSTEM ARCHITECTURE & DATA PIPELINE

# Chapter 3: Component Hierarchy & Reactive UI State Machine

## 3.1 Master Controller Architecture ( `YolkFlowClient.tsx` )

The client application is anchored by `YolkFlowClient.tsx` (5,550+ lines of production TypeScript). It orchestrates reactive state across three primary operational views:

1. **Dashboard View ( `activeTab === "dashboard"` ):** Executive business intelligence view providing real-time KPI metrics, rolling margin calculations, SVG spline trend charts, per-item egg sales volume breakdowns, and a sortable historical sales ledger table.
2. **Daily Entry View ( `activeTab === "entry"` ):** Interactive 4-panel digitization interface mirroring the physical Hal-Khata page. Includes stock valuation, credit collections, opening liabilities, and expense rows with real-time mathematical validation.
3. **Overhead & Savings View ( `activeTab === "overhead"` ):** Enterprise operational cost tracker ( `OverheadExpensesView.tsx` ) managing 11 expense categories, shop reserve funds, bank DPS, and personal owner withdrawals.

## 3.2 Component Relationship & Modal Hierarchy

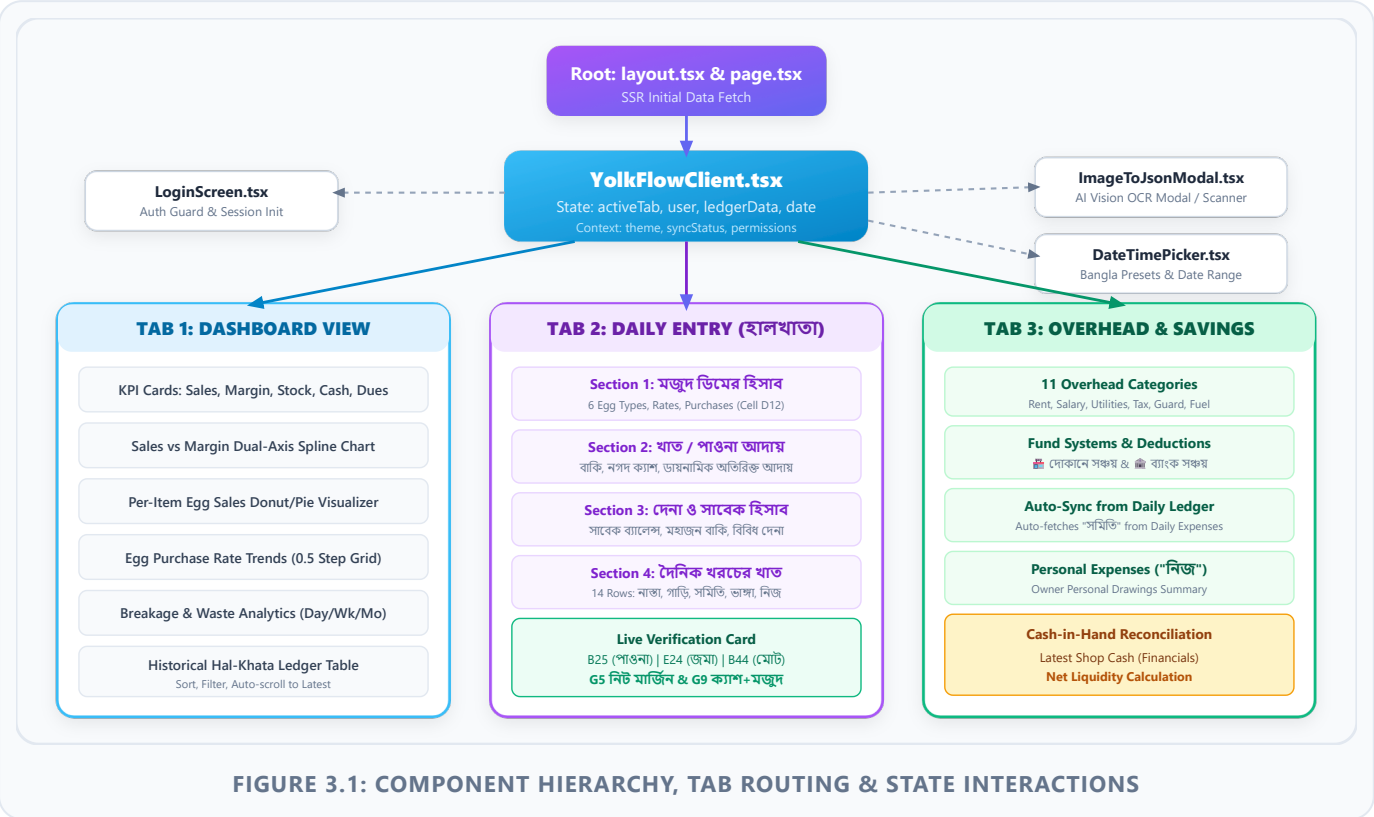



FIGURE 3.1: COMPONENT HIERARCHY, TAB ROUTING & STATE INTERACTIONS

## 3.3 Reactive State Synchronization

When the user navigates between dates, `YolkFlowClient` synchronizes state using the `selectedDashboardDate` and `currentViewDay` state hooks. If an existing entry is found, the system switches to *Edit Mode*, hydrating all 6 egg stock

inputs, dynamic extra collections, supplier dues, and expense rows. When creating a new day, closing stock from the previous date carries forward automatically as opening inventory.

# Chapter 4: Dual-Layer Persistence & Cloud Synchronization Engine

## 4.1 Zero-Downtime Hybrid Storage Strategy

A critical engineering requirement of YolkFlow is offline resilience. Wholesale egg depots may experience intermittent internet connectivity. To solve this, YolkFlow implements a dual-layer persistence engine inside `src/lib/db.ts` and `src/lib/googleSheets.ts`:

```
Primary Cloud: Google Sheets API v4 (Service Account Key) Secondary Local: process.cwd() /
"mock_db.json" (Atomic Sync)
```

## 4.2 The Read Pipeline (`getDatabaseData`)

The system first calls `isGoogleSheetsConfigured()`, checking for the existence of `GOOGLE_SHEET_ID`, `GOOGLE_SERVICE_ACCOUNT_EMAIL`, and `GOOGLE_PRIVATE_KEY`. If configured, it performs a parallel read across three master worksheets: `DailyStock`, `Financials`, and `Expenses`. Rich metadata (such as dynamic JSON arrays for extra dues and collections) is merged with local cache. If network failure occurs, the engine falls back instantly to `mock_db.json`.

## 4.3 The Write & Sheet Duplication Pipeline (`saveLedgerEntry`)

When saving an entry, YolkFlow writes to the local JSON file first, ensuring zero data loss. Next, it updates the master Google Sheets tabs via `updateSheetValues()`. Finally, it executes `createOrUpdateDayTab()`:

- Converts the date (e.g. `2026-08-25`) to tab format: `25/08/26`.
- Inspects spreadsheet metadata. If the tab does not exist, it duplicates the pre-formatted `Original` template sheet.
- Injects **native spreadsheet formulas** (e.g. `=B6*C6`, `=SUM(B6:B11)`, `=B44-E24`) into exact cells, allowing human accountants to verify math directly inside Google Sheets.

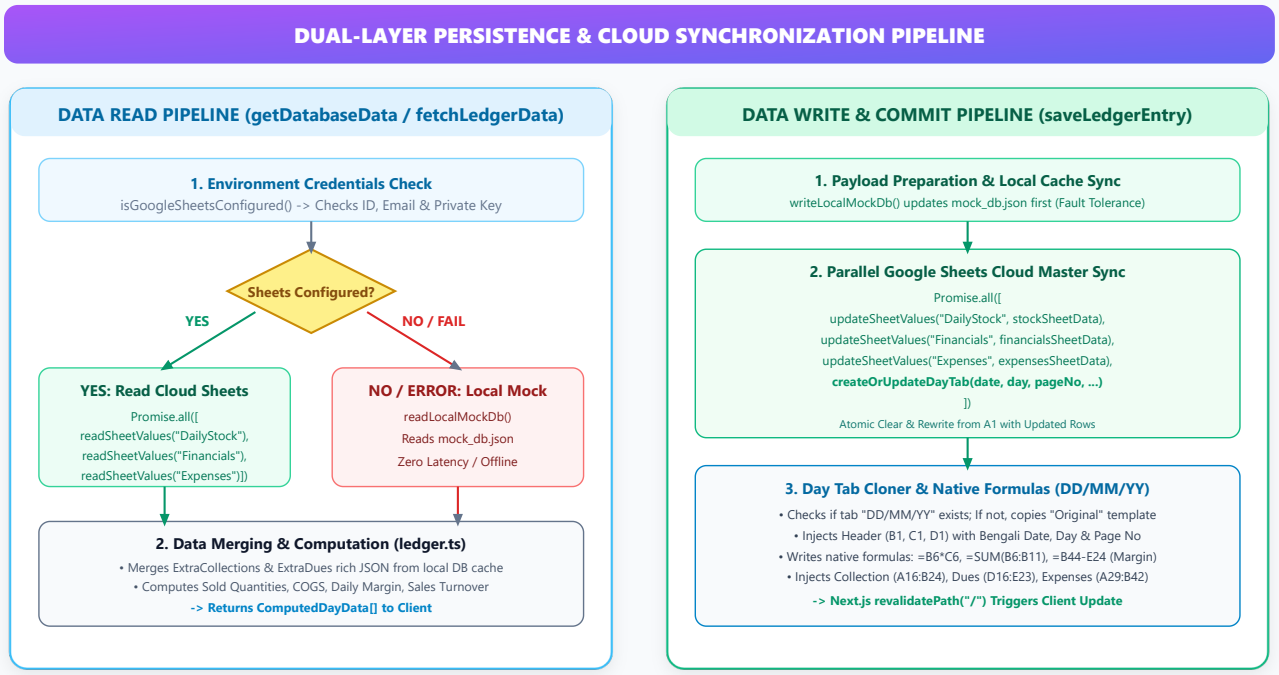



FIGURE 4.1: DUAL-LAYER READ/WRITE PIPELINE & DAILY TAB CLONER

# Chapter 5: Entity-Relationship Modeling & Data Schema

## 5.1 Relational Structure on Flat-File Storage

Although stored in Google Sheets and JSON, YolkFlow maintains a strictly typed relational schema defined in `src/lib/db.ts` and `src/lib/googleSheets.ts`. The primary temporal foreign key across all tables is the ISO date string ( `YYYY-MM-DD` ).

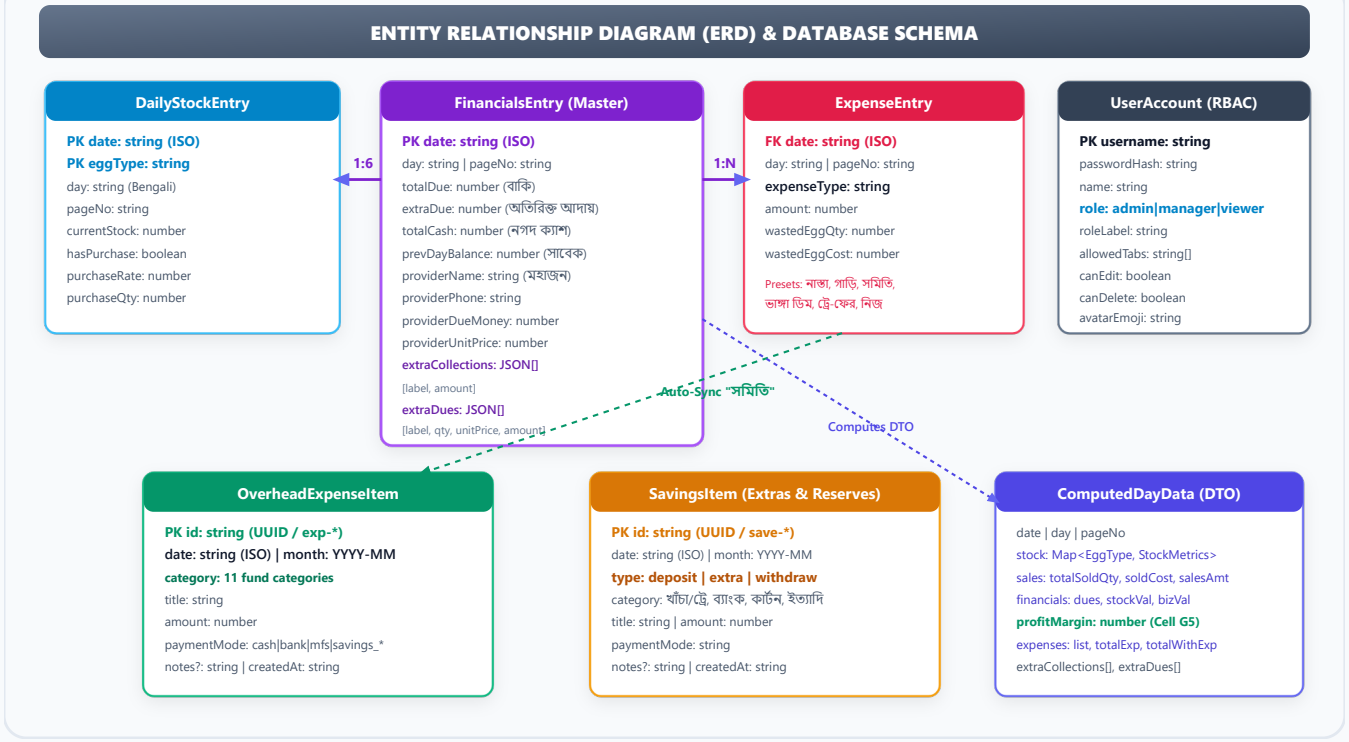



FIGURE 5.1: ENTITY-RELATIONSHIP DIAGRAM (ERD) & TYPESCRIPT DATA MODELS

## 5.2 Data Dictionary

| Entity Name | Primary Key | Foreign Key | Key Attributes & Types |
|---|---|---|---|
| **DailyStockEntry** | date + eggType | date | currentStock: number, hasPurchase: bool, purchaseRate: number, purchaseQty: number |
| **FinancialsEntry** | date | date | totalDue: number, totalCash: number, prevDayBalance: number, extraCollections: JSON, extraDues: JSON |
| **ExpenseEntry** | date + expenseType | date | expenseType: string, amount: number, wastedEggQty: number, wastedEggCost: number |
| **OverheadExpenseItem** | id (UUID) | date | month: YYYY-MM, category: 11 fund types, title: string, amount: number, paymentMode: string |
| **SavingsItem** | id (UUID) | date | type: deposit\|extra\|withdraw, category: crate/carton/bank, amount: number |
| **UserAccount** | username | N/A | role: admin\|manager\|viewer, allowedTabs: string[], canEdit: bool, canDelete: bool |

# Chapter 6: Hal-Khata Mathematical Accounting Engine

## 6.1 The Traditional Bengali Hal-Khata Accounting Equation

The mathematical heart of YolkFlow resides in `src/lib/ledger.ts`. It replicates the exact accounting equation used by Bengali wholesale egg merchants for over half a century. The equation balances four distinct tables:

```
১. মজুদ ডিমের মূল্য (Cell D12) = ∑ (CurrentStock_i × PurchaseRate_i) ২. সর্বমোট পাওনা/হিসাব (Cell B25) = TotalDue + ExtraDue + TotalCash + D12 ৩. মোট খরচ (Cell B43) = ∑ ExpenseAmount_j ৪. দেনা ও জমার মোট (Cell E24) = PrevDayBalance + ProviderDueMoney + ∑ ExtraDues
──────────────────────────────────────────────────────────────── সর্বমোট সমীকরণ (Cell B44) = Cell B25 (সর্বমোট পাওনা) + Cell B43 (মোট খরচ) 👑 দৈনিক নিট লাভ / মার্জিন (Cell G5) = Cell B44 - Cell E24
```

## 6.2 Derivation of Daily Sales & Stock Outflow

Since wholesale eggs sell continuously without individual barcode scans, sold volume is computed by solving the inventory balance equation:

```
SoldQty_i = Max(0, PrevStock_i + PurchaseQty_i - CurrentStock_i) Cost of Goods Sold (COGS) = ∑ (SoldQty_i × ActivePurchaseRate_i) Daily Sales Turnover (BDT) = COGS + Max(0, ProfitMargin)
```

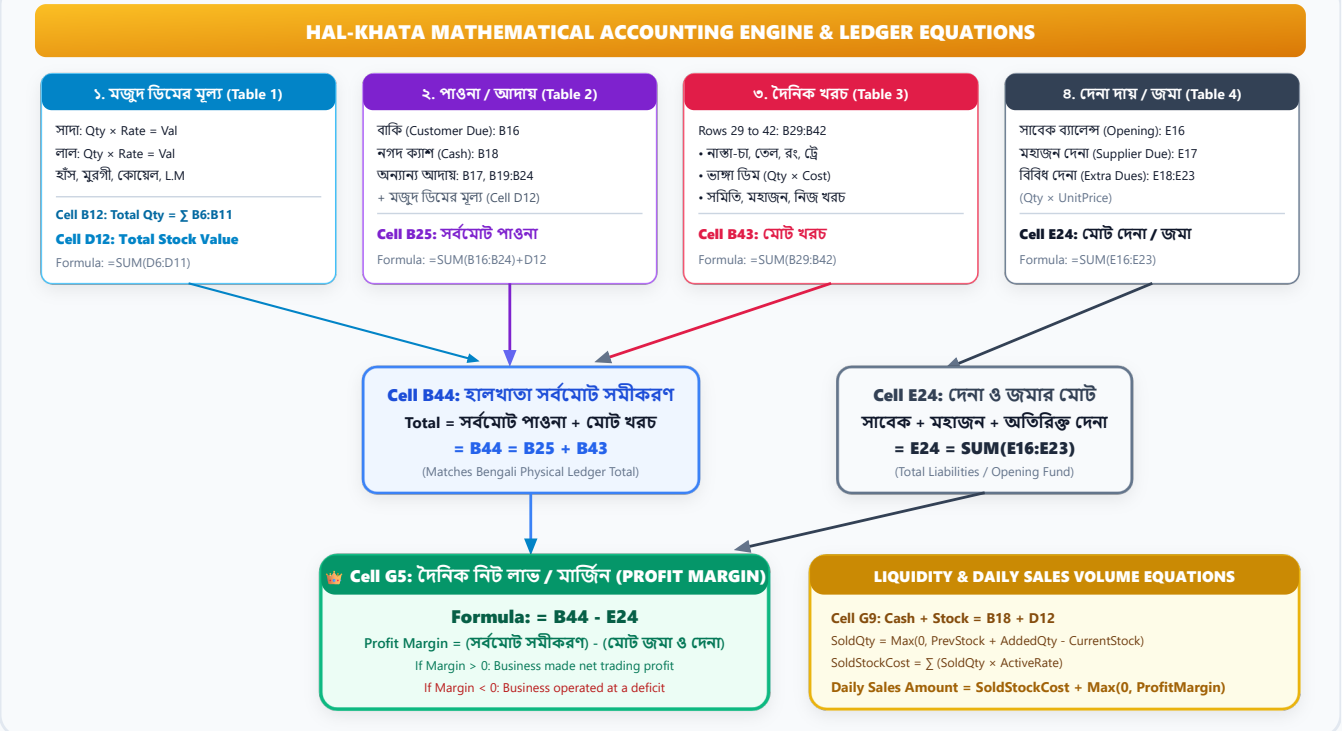



FIGURE 6.1: HAL-KHATA MATHEMATICAL FLOW & CELL G5 PROFIT MARGIN CALCULATION

# Chapter 7: Daily Transaction Lifecycle & Hal-Khata Entry Workflow

## 7.1 State Machine: From Physical Page to Ledger Commit

The transaction lifecycle begins in `YolkFlowClient.tsx` when the accountant selects a business date. The state machine follows four disciplined phases:

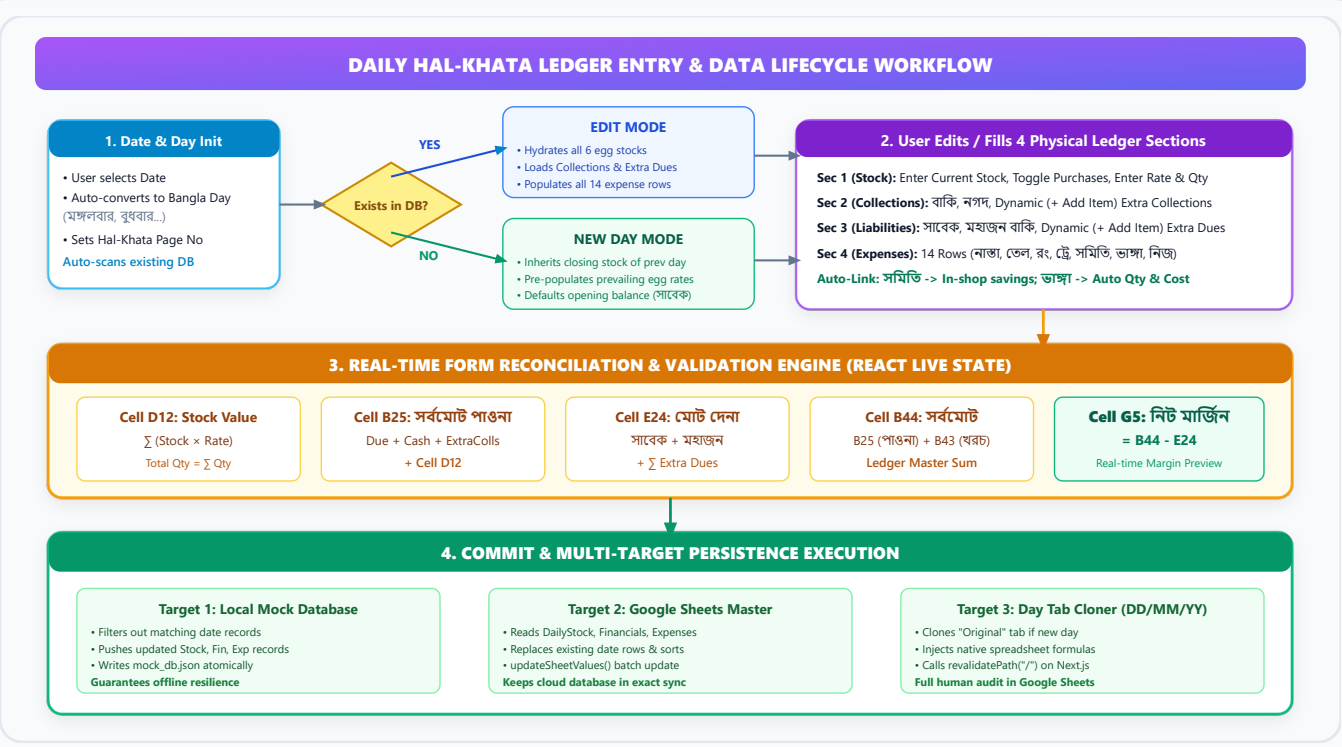



FIGURE 7.1: DAILY ENTRY STATE MACHINE, LIVE CALCULATIONS & PERSISTENCE EXECUTION

## 7.2 Phase Breakdown

- **Phase 1: Date Initialization & Auto-Bangla Day:** The system takes an ISO date (e.g. `2026-08-25`), computes the day of the week, and maps it via `BANGLA_DAYS_MAP` to Bengali (`মঙ্গলবার`).
- **Phase 2: Hydration & Carryover:** If records exist, the form populates in Edit Mode. If entering a new day, the system fetches the closing stock and purchase rates from the immediate prior ledger day.
- **Phase 3: Live Verification Card:** As numbers are typed into Section 1 (Stock), Section 2 (Collections), Section 3 (Dues), or Section 4 (Expenses), the client re-computes B25, E24, B43, B44, and G5 in real-time, displaying green indicators if balanced.
- **Phase 4: Multi-Target Commit:** `saveLedgerEntryAction()` atomically updates the local JSON database, writes to Google Sheets master tabs, and clones a daily sheet tab with native formulas.

# Chapter 8: AI Vision OCR Ingestion Pipeline

## 8.1 Ingesting Handwritten Bengali Notebook Pages

One of YolkFlow's most innovative features is its AI Vision OCR engine implemented in `src/app/ImageToJsonModal.tsx` and `src/app/api/ocr-to-json/route.ts`. It enables merchants to photograph a physical Hal-Khata page and transform it into structured JSON with 100% mathematical accuracy.

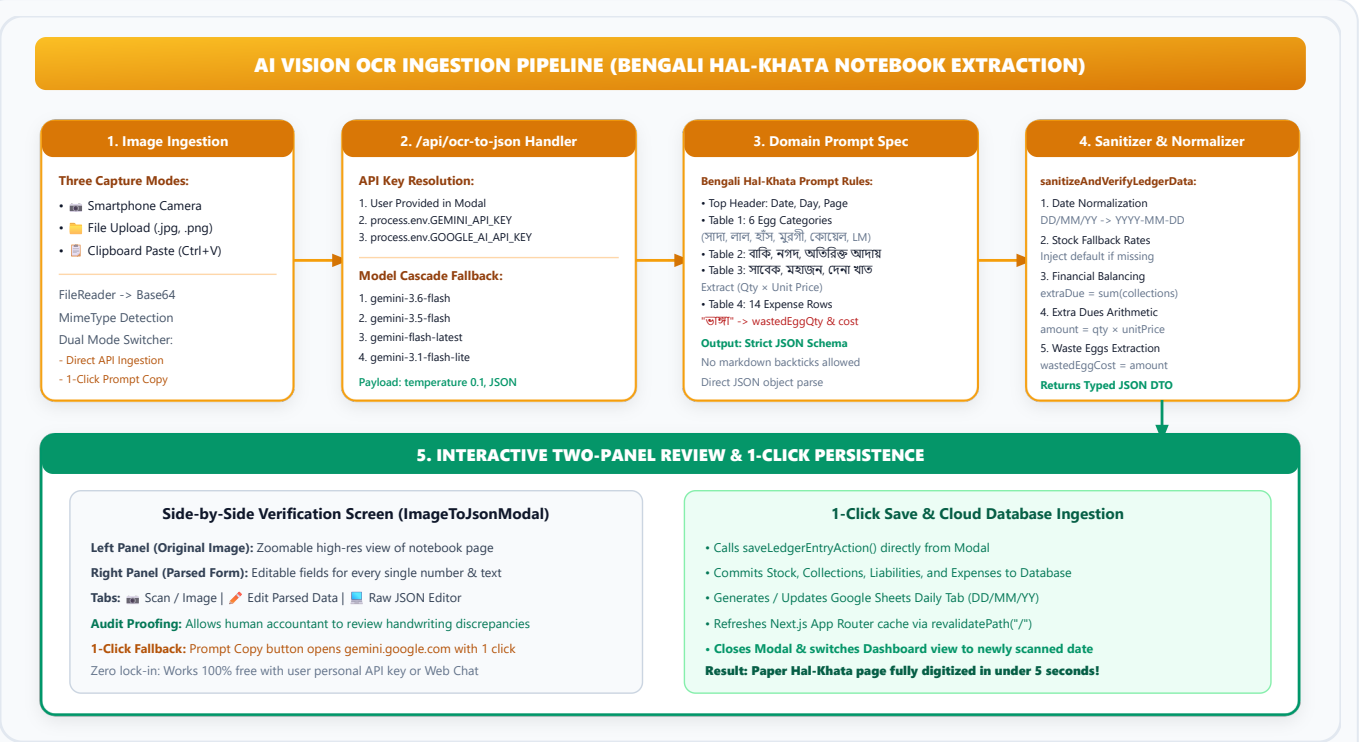



FIGURE 8.1: GEMINI VISION OCR ARCHITECTURE & HANDWRITING EXTRACTION PIPELINE

## 8.2 Multi-Model Cascade & Fallback Resilience

To guard against Google Gemini rate limits or transient API outages, the route handler executes a graceful cascade through four candidate models:

1. `gemini-3.6-flash` (Default high-speed vision model)
2. `gemini-3.5-flash` (Secondary fallback)
3. `gemini-flash-latest` (General production fallback)
4. `gemini-3.1-flash-lite` (High-throughput lightweight model)

## 8.3 The Domain Extraction Prompt & Sanitizer (`sanitizeAndVerifyLedgerData`)

The system prompt instructs Gemini to read the four standard Hal-Khata sections, enforcing ISO date formatting, six standard egg categories, and broken egg extraction ("ভাঙ্গা" expenses). The sanitizer validates the extracted JSON, recalculating amounts and checking that no egg category is omitted.

# Chapter 9: Overhead Expenses, Fund Settlements & Liquidity Flow

## 9.1 The 11 Overhead Categories & Fund Channels

In wholesale trading, operating expenses must be cleanly segregated from inventory trading costs. `OverheadExpensesView.tsx` and `/api/overhead` manage 11 distinct operational cost categories organized into four financial channels:

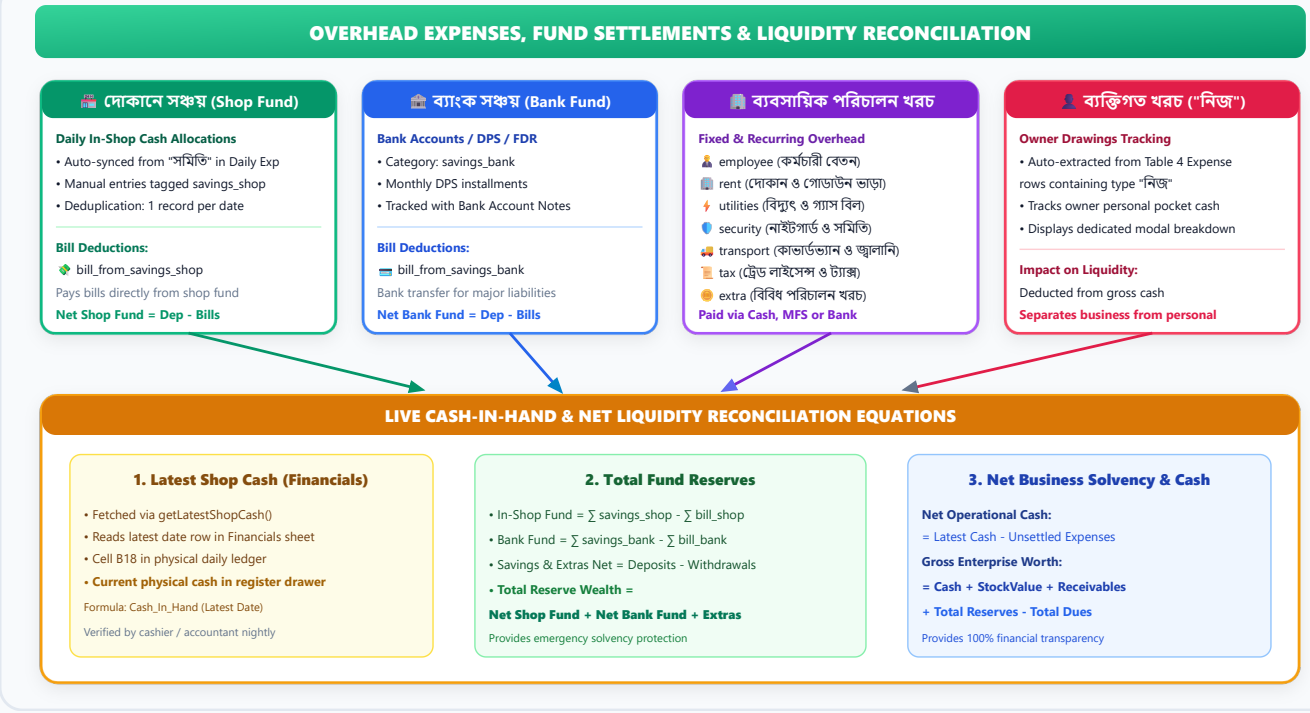



FIGURE 9.1: OVERHEAD FUND CHANNELS, SAMITY SYNC & CASH RECONCILIATION

## 9.2 Key Fund Integrations

- **Automated "সমিতি" Sync:** In Bengali markets, merchants make daily cooperative savings ("সমিতি"). When entered in Table 4 of the daily ledger, the system automatically imports this entry into the *দোকানে সঞ্চয়* (In-Shop Savings) fund.
- **Fund-Linked Bill Reductions:** When recurring operating bills (e.g. employee bonuses or shop repairs) are paid from accumulated funds, selecting `bill_from_savings_shop` or `bill_from_savings_bank` deducts the amount from the reserve balance rather than depleting the daily cash register.
- **Personal Withdrawals Tracking ("নিজ"):** Personal drawings taken by the business owner from cash receipts are flagged with the preset "*নিজ*". The system aggregates these separately, ensuring business liquidity remains distinct from personal household expenditures.

# Chapter 10: Role-Based Access Control (RBAC) & Security

## 10.1 Multi-Tier Authorization Model ( `src/lib/auth.ts` )

YolkFlow implements an enterprise security model with three standard roles: **Admin (Owner)**, **Manager (Accountant)**, and **Viewer (Auditor/Guest)**.

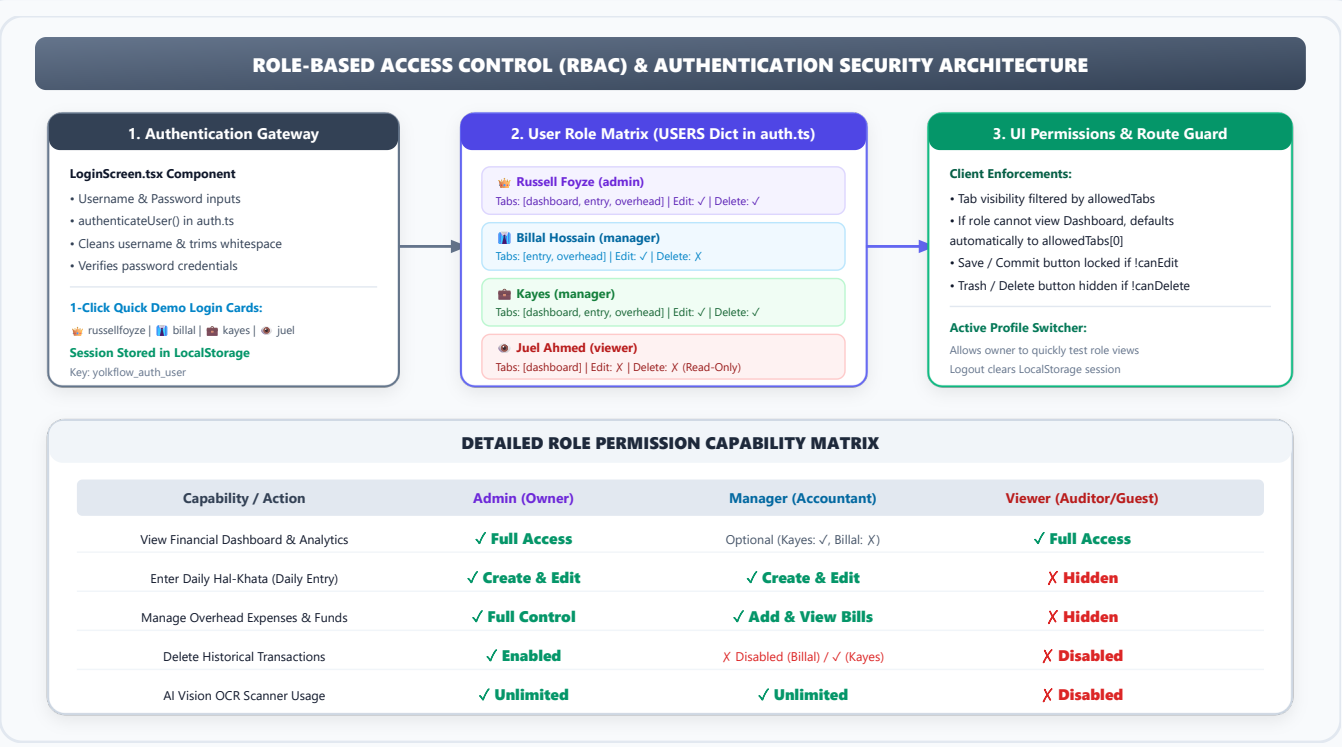



FIGURE 10.1: RBAC SECURITY ARCHITECTURE & USER PERMISSION MATRIX

## 10.2 User Account Profiles

| Username | Name & Title | Role | Allowed Tabs | Can Edit | Can Delete |
|---|---|---|---|---|---|
| **russellfoyze** | Russell Foyze ( 👑 মালিক / অ্যাডমিন) | admin | dashboard, entry, overhead | ✓ Yes | ✓ Yes |
| **billal** | Billal Hossain ( 👔 হিসাবরক্ষক) | manager | entry, overhead | ✓ Yes | ✗ No |
| **kayes** | Kayes ( 💼 ম্যানেজার) | manager | dashboard, entry, overhead | ✓ Yes | ✓ Yes |
| **juel** | Juel Ahmed ( 👁️ ভিউয়ার) | viewer | dashboard | ✗ No | ✗ No |

# Chapter 11: Real-time Business Intelligence & Analytics Engine

## 11.1 Dynamic Vector Spline Charts ( `getSplinePath` )

Rather than relying on bloated third-party charting libraries, YolkFlow generates pure, lightweight, responsive SVG charts directly in React. The **Sales vs Margin Trend Chart** uses Catmull-Rom spline interpolation ( `getSplinePath` ) to compute smooth cubic Bézier curves.

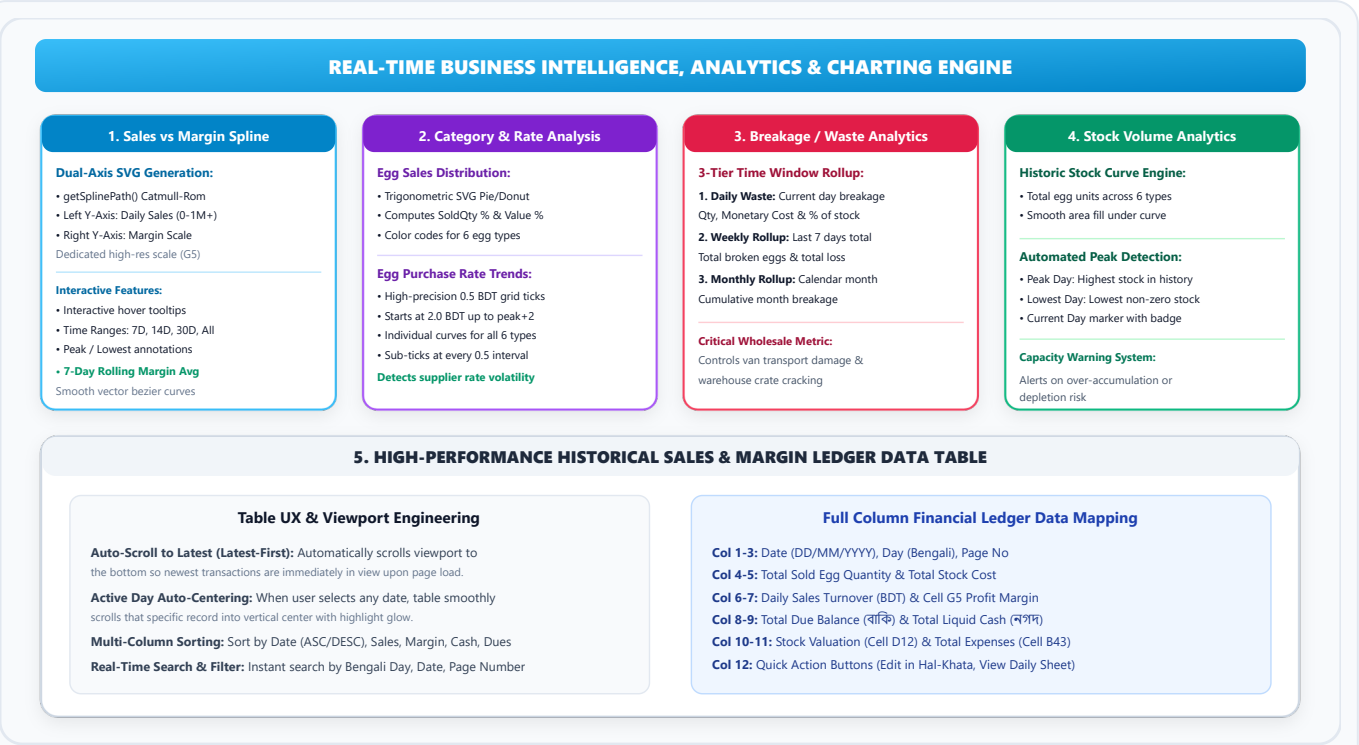



FIGURE 11.1: ANALYTICS ENGINE, CHART SPLINES & WASTE ROLLUP TIERS

## 11.2 Key Analytical Visualizers

- **Dual Y-Axis Spline Engine:** Visualizes Daily Sales Volume on the primary left axis (scale up to 1,000,000+ BDT) while plotting Profit Margin (Cell G5) on a dedicated, high-resolution right axis.
- **0.5 BDT Grid Egg Price Trend:** Tracks procurement rates for all 6 egg categories across daily entries with 0.5 BDT increment gridlines, exposing supplier rate volatility.
- **3-Tier Waste & Breakage Rollup:** Aggregates broken egg expenses across Daily, 7-Day Rolling Weekly, and Calendar Month windows to compute breakage percentage against total inventory.
- **High-Performance Sales Ledger Table:** Features auto-scrolling to the latest transaction, date centering, multi-column sorting, and Bengali keyword search.

# Chapter 12: Production Readiness, Fault Tolerance & Recommendations

## 12.1 Reliability & Fault Tolerance Assessment

YolkFlow demonstrates remarkable operational resilience due to its hybrid architecture:

- **Air-Gapped Offline Capability:** If Google Cloud experiences an outage or internet fails at the depot, the application functions seamlessly on `mock_db.json` without throwing UI crashes.
- **Human-in-the-Loop AI:** The OCR scanner never writes directly to the database without manual accountant verification, eliminating OCR hallucination risk.
- **Cloud Spreadsheet Auditability:** By duplicating daily tabs with native spreadsheet formulas, non-technical business partners can verify calculations using Google Sheets on mobile devices.

## 12.2 Architectural Recommendations for Future Scaling

1. **PostgreSQL / Supabase Migration (at > 50,000 rows):** While Google Sheets serves perfectly as a lightweight database up to several thousand rows, migrating historical archives to PostgreSQL will maintain sub-second response times as data expands over decades.
2. **Automated SMS / WhatsApp Daily Summary:** A scheduled cron job triggering nightly at 11:00 PM to send the owner an automated WhatsApp summary of Cell G5 Profit Margin, Cash in Hand, and Closing Egg Stock.
3. **Bluetooth Thermal Receipt Printing:** Direct integration with 58mm/80mm Bluetooth thermal printers for generating customer due memos directly from the Hal-Khata entry screen.

# Appendix: Reference Index

## A. Mathematical Equation Reference

| Metric Name | Physical Ledger Cell | Exact Code Formula |
|---|---|---|
| Total Stock Quantity | Cell B12 | `∑ (stockEntries[i].currentStock)` |
| Total Stock Value | Cell D12 | `∑ (stockEntries[i].currentStock × stockEntries[i].purchaseRate)` |
| Total Receivables / পাওনা | Cell B25 | `totalDue + extraDue + totalCash + totalStockValue (D12)` |
| Total Operating Expenses | Cell B43 | `∑ (expensesList[j].amount)` |
| Total Liabilities / জমা | Cell E24 | `prevDayBalance + providerDueMoney + ∑ extraDues[k].amount` |
| Grand Balance Equation | Cell B44 | `Cell B25 (সর্বমোট পাওনা) + Cell B43 (মোট খরচ)` |
| **Net Profit Margin** | **Cell G5** | **`Cell B44 (সর্বমোট সমীকরণ) - Cell E24 (মোট দেনা ও জমা)`** |
| Cash + Stock Assets | Cell G9 | `totalCash + totalStockValue (Cell B18 + Cell D12)` |
| Sold Egg Units | Derived | `Max(0, PrevStock + AddedQty - CurrentStock)` |
| Cost of Goods Sold (COGS) | Derived | `∑ (SoldQty_i × ActivePurchaseRate_i)` |
| Daily Sales Turnover | Derived | `COGS + Max(0, ProfitMargin)` |

## B. Google Sheets Master Worksheets Specification

| Worksheet Tab | Primary Responsibility | Headers (Columns A to Z) |
|---|---|---|
| **DailyStock** | Master stock log | Date, Day, PageNo, EggType, CurrentStock, HasPurchase, PurchaseRate, PurchaseQty |
| **Financials** | Master finance log | Date, Day, PageNo, TotalDue, ExtraDue, TotalCash, PrevDayBalance, ProviderName, Phone, EggType, DueMoney, UnitPrice, ExtraCollectionsJSON, ExtraDuesJSON |
| **Expenses** | Master expenses log | Date, Day, PageNo, ExpenseType, Amount, WastedEggQty, WastedEggCost |
| **OverheadExpenses** | Operating overheads | Id, Date, Month, Category, Title, Amount, PaymentMode, Notes, CreatedAt |
| **SavingsAndExtras** | Reserves & extra income | Id, Date, Month, Type, Category, Title, Amount, PaymentMode, Notes, CreatedAt |
| **DD/MM/YY Tabs** | Daily Hal-Khata clone | Full Bengali formatted layout with native formulas =B6*C6, =B44-E24, etc. |

End of Specification Document • YolkFlow Architecture Compendium • Generated October 2026